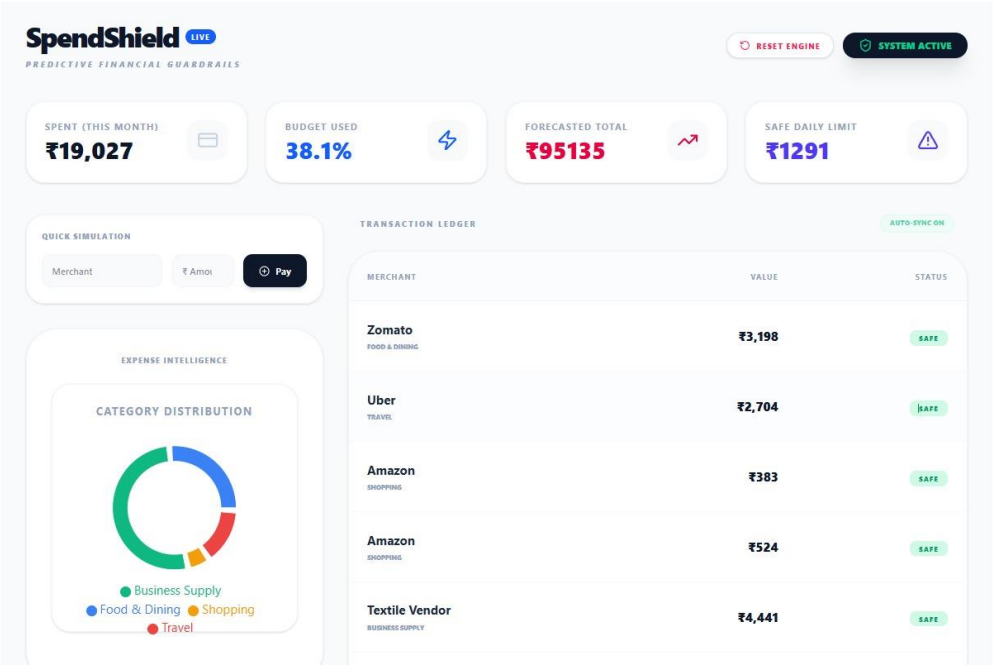
Solution & Design Approach

Solution Overview

The proposed system, **SpendShield**, is a designed to help users monitor, control, and optimize their spending behavior.

Unlike traditional payment applications that only display transaction history, SpendShield provides **proactive control** by analyzing transactions in real time and generating alerts before users exceed their budget limits.

The system integrates multiple digital payment sources and applies intelligent logic to categorize, evaluate, and guide user spending decisions.



Design Approach

SpendShield: Design and Approach

1. System Architecture: The Decoupled Pipeline

We implemented a **Layered and Modular Architecture** to ensure sub-millisecond inference and real-time scalability. By decoupling the data ingestion from the UI, we ensure the system remains responsive even during high-frequency transaction bursts.

Data Collection Layer (Simulator):** Uses `stream.js` to mock unified UPI/Card/Wallet APIs.

Processing Layer (Backend):** A Node.js/Express environment that acts as the "Feature Store."

Data Storage Layer (Persistence):** MongoDB Atlas using **Time-Series Indexing** on `userId` and `date` for high-speed retrieval.

Presentation Layer (Frontend):** A React dashboard utilizing **Parallel Data Fetching** (`Promise.all`) to reduce visual latency.

2. The Hybrid Inference Engine (The "Big Three")

Unlike traditional apps that just show history, SpendShield uses **Deterministic Inference**—transpiling mathematical weights directly into the backend for zero-latency feedback.

A. Linear Regression ($y = mx$) | Velocity Forecast**

Function: Projects monthly spending based on current "Burn Rate."

Example:On April 10th, if Arjun has spent **₹15,000**, the engine identifies a velocity of **₹1,500/day**. It forecasts a month-end total of **₹45,000**, allowing the "Safe Daily Cap" to remain flexible.

B. Decision Tree Classifier | Contextual Guardrails

Function: Uses a Root → Decision → Leaf structure to categorize spending.

Example:A ₹22,000 transaction at "Surat Textile Hub" hits the system. The tree splits by *Amount > ₹15k* and *Merchant Type*. It identifies it as **"Business Inventory"** and stays **SAFE**, preventing a false "Overspend" alert.

C. Temporal Sequence Analysis (LSTM-lite) | Anomaly Detection

Function: Analyzes the Sequence of the last 5 transactions.

Example: Arjun usually spends ₹200 on tea. Suddenly, an ₹8,000 cafe bill appears. The engine detects a 1,500% jump from his rolling mean and instantly flips the status to WARNING, shrinking his daily limit to "punish" the overspend.

3. Technical Implementation & Scalability

Sub-millisecond Inference: By porting ML logic to Node.js, we avoid the overhead of calling external Python microservices.

Stateless Reliability: The backend is stateless; it recalculates the user's financial standing from the MongoDB "Truth" on every request.

Account Aggregator (AA) Ready: While using a simulator today, the schema is 100% compatible with Sahamati/Open Banking APIsfor real-world read-only consent.

Hybrid Approach: We chose Heuristics + Statistics over "Black-Box" Neural Networks to ensure Arjun can actually understand why the AI is telling him to stop spending.

```
spendshield/
├── client/                # React Frontend (Vite + Tailwind)
│   ├── src/
│   │   ├── components/    # UI Components (StatCard, Chart, HistoryTable)
│   │   ├── App.jsx        # Dashboard Logic & Live Sync Engine
│   │   └── main.jsx
│   └── tailwind.config.js # Design System Tokens
├── server/                # Node.js Backend & ML Engine
│   ├── server.js          # Hybrid ML Inference API & MongoDB Logic
│   ├── stream.js          # Real-time Transaction Ingestion Simulator
│   └── .env               # Configuration (Mongo URI & Ports)
├── README.md              # Project Documentation
└── package.json           # Global dependencies
```